Contents

CreditSetu – Live Demo Script **1**
Credentials used in this script . . . 1
1. Landing page – real borrower/SHG/lender stories . . . 2
2. Borrower login – independent borrower, cold start . . . 2
3. Borrower login – SHG-linked borrower, cold start . . . 2
4. Borrower login – mature borrower, model-driven score + SHAP . . . 2
5. Marketplace – request a loan targeted at a specific lender . . . 2
6. Switch to the lender – see the request live, approve it . . . 3
7. Back to the borrower – confirm it reflects . . . 3
8. Repayment – dummy payment gateway, per-loan wallet check . . . 3
9. Score history – chronological reading . . . 3
10. SHG dashboard – group oversight . . . 3
11. Admin – platform oversight only . . . 3
12. Dark mode and accessibility . . . 4

CreditSetu – Live Demo Script

Note on scope: this script walks the live application (http://localhost:5173) as it behaves today. The payment flow described in Step 8 is **per-loan**: the borrower's wallet balance is checked against each loan's own due amount individually, not against the sum of all active loans at once. If you are also showing the Midterm-Prototype-Report.pdf, its section 4.7 / Table 19 / Figure 9 describe an earlier *aggregate*-blocking version of this flow – call that out if asked, rather than letting the two appear to disagree silently.

Important – single session per browser. Only one role (borrower / lender / SHG / admin) can be logged in per browser at a time, by design (the product requirement was separate, never-combined logins). Demo the borrower -> lender -> borrower flow **sequentially in one browser window** (log out, then log back in as the next role) rather than relying on two tabs of the same browser open side by side. If you do want two roles visible at once, open the second one in an incognito/private window instead.

Shared demo password for every borrower/lender/SHG account: `password123`.

Credentials used in this script

Role	Login	Password	Notes
Borrower (independent, cold start)	phone `7583094515`	`password123`	Praneel Anne – score 450/Poor
Borrower (SHG-linked, cold start)	phone `1656670106`	`password123`	Niharika Gupta – score 553/Fair, Isaac Vikas Group
Borrower (mature, model + SHAP)	phone `5560571219`	`password123`	Aarush Pant – score ~890s/Excellent
Lender	username `district-cooperative-bank`	`password123`	District Cooperative Bank
SHG	username `isaac-vikas-group`	`password123`	Isaac Vikas Group – Niharika's SHG
Admin	username `admin`	`admin123`	Platform oversight only

1. Landing page – real borrower/SHG/lender stories

- Open the site at `/`. Point out the home page is built entirely from data already in the system (no fabricated testimonials): borrower spotlights, SHG spotlights, and lender spotlights are all pulled live from the database via `/api/dashboard/*` endpoints.
- Mention the four separate logins visible from the header – Borrower, Lender, SHG, Admin – each a fully separate session/table, so a borrower's token can never read lender or SHG data.

2. Borrower login – independent borrower, cold start

- Log in as **Praneel Anne**, phone `7583094515`, password `password123`.
- Praneel has no SHG link (`is_shg_linked: false`) and no loans yet. His dashboard shows:
  - Score **450**, risk label **Poor** (the `independent_bootstrap` base component).
  - A single score-history point and plain-language explanation: "Independent starting score (no SHG link)" is worth 150 of his 450 points.
  - This demonstrates the dual-path cold-start design: an independent borrower starts flat at `INDEPENDENT_BASE_SCORE = 450` until enough real repayment events accumulate.

3. Borrower login – SHG-linked borrower, cold start

- Log in as **Niharika Gupta**, phone `1656670106`, password `password123`.
- Niharika is linked to **Isaac Vikas Group** SHG (`shg_bootstrap` base component).
- Her starting score is **553** (risk label **Fair**) – noticeably higher than Praneel's 450, because her score inherits a share of her SHG's track record (group repayment/attendance rates) rather than starting completely neutral. This is the "bootstrap head start" for group-linked borrowers mentioned in the proposal's scoring design.
- Open her SHG page from the dashboard to show group-level stats (member count, aggregate repayment rate, aggregate attendance rate) computed with a single `GROUP BY` SQL query (the N+1 fix) rather than walking every member's record in Python.

4. Borrower login – mature borrower, model-driven score + SHAP

- Log in as **Aarush Pant**, phone `5560571219`, password `password123`.
- Aarush has 3 active loans and 11 repayment events on record, so his score has fully transitioned off the cold-start blend onto the trained XGBoost model (`base_component: "model"`).
- Current score: **892**, risk label **Very Low** – shown on-screen as **Excellent** (the credit-bureau-style label mapping, not the raw internal "Very Low" risk string, which read confusingly as if the score itself were bad).
- Open his score explanation panel: this is the live SHAP breakdown, feature by feature – `missed_rate` (+71.3), `avg_days_late` (+37.7), `on_time_rate` (+32.0), `savings_regularity` (+25.7), down to small negative contributors like `savings_consistency` (-23.0) and `n_closed_loans` (-12.4). Point out this is model explainability in production, not a canned message – every number changes if his repayment pattern changes.
- Scroll to "Ways to improve your score" – personalized, generated from the same SHAP values (e.g. "irregular savings is pulling your score down").

5. Marketplace – request a loan targeted at a specific lender

- From a borrower dashboard (e.g. Praneel's), open "Request a loan." Fill in principal, tenure, an optional purpose, then switch the radio from "Broadcast to all eligible lenders" to **"Choose a specific lender"** and pick **District Cooperative Bank**. Submit.
- The request now appears instantly, below, in "Your loan requests" as **Pending**, targeted at District Cooperative Bank specifically (not broadcast) – this targeting (`LoanRequest.target_lender_id`) was a gap found during earlier live testing (a borrower couldn't choose who saw their request) and was added as a lender picker on the form.

6. Switch to the lender – see the request live, approve it

- Log out. Log in as the lender: username `district-cooperative-bank`, password `password123`.
- **Incoming loan requests** – the request just submitted appears immediately in this lender's queue (and only this lender's, since it was targeted).
- While here, also show:
  - **Reputation score** – a marketplace ranking badge ("Rank 3 of 6"), tied to the lender-incentive system (better rates / more activity raise the rank).
  - **Your linked SHGs** – Approved / Pending-incoming / Pending-outgoing partnership states, with Approve/Reject actions for incoming requests.
- Set a rate and click **Approve** on the new request. Status flips to Approved in place.

7. Back to the borrower – confirm it reflects

- Log out, log back in as the same borrower. The request now shows **Approved – District Cooperative Bank**, and the new loan appears immediately in "Your loan history" as Active with the rate the lender set. This closes the loop: borrower request -> lender queue -> lender decision -> borrower's own view, all through the real API, no manual refresh trick needed as long as you're logged in as the role you're viewing (see the single-session note above if a second tab was also open).

8. Repayment – dummy payment gateway, per-loan wallet check

- From Aarush's dashboard, click "Repay a loan." This opens the dummy payment gateway flow:
  1. **Gateway login screen** – fake credential form (username `abc`, password `123`), purely cosmetic, to give the "redirected to a payment provider" feel without a badge announcing it's a demo.
  2. **Buffering screen** – a ~1.4s "Connecting to payment gateway…" delay.
  3. **Loan list with wallet balance** – shows the borrower's wallet balance and each active loan's amount due this cycle.
- Key behavior to demonstrate: each loan row is independently payable or not based on whether the wallet balance covers *that loan's* due amount – **not** the sum of all due amounts. Pay one loan that the wallet covers even if the total across all loans exceeds the balance; the other loan(s) stay untouched and still payable on their own merits. (This was changed from an earlier aggregate-blocking version after live testing showed it was confusing – a wallet that covered one loan was blocking repayment of all of them.)
- After a successful payment, show the on-time/late status chip and the new score, computed immediately from the repayment event.

9. Score history – chronological reading

- Back on the dashboard, point at the score-history line chart: dates run left-to-right, oldest to newest, like any normal time series (this was flipped backwards earlier and fixed by removing an unnecessary `.reverse()` – the backend already returns the data in ascending order).

10. SHG dashboard – group oversight

- Log out, log in as the SHG: username `isaac-vikas-group`, password `password123`.
- Show the member list (including Niharika from Step 3), attendance and savings records, and the SHG's own view of pending lender-partnership decisions – an SHG can also initiate a link request to a lender, not just respond to one (`initiated_by_role`).

11. Admin – platform oversight only

- Log out, log in as admin: username `admin`, password `admin123` (env-configured, no separate DB row/session table entry the way borrowers/lenders/SHGs have).

- Show the admin dashboard is oversight-only: platform-wide stats, SHG-lender link management (approve/reject), no ability to directly edit an individual borrower's data or manually recompute a score outside the normal repayment-driven flow.

12. Dark mode and accessibility

- From the header utility bar, toggle **Dark mode**, **High contrast**, and the text-size controls. Mention every *fresh* page load defaults back to light mode regardless of system theme or the previous session's choice – a deliberate default, not a bug.